

Федеральное государственное автономное образовательное учреждение высшего
образования «Национальный исследовательский университет ИТМО»

Факультет Программной Инженерии и Компьютерной Техники

Отчёт

по дисциплине «Разработка мобильных приложений»

Выполнили:

Бондаренко Дарья Александровна,
Волкова Ирина Александровна,
Гавриш Богдан Вячеславович,
Галиева Аделина Наилевна,
Давлетов Тимур Маратович,
Старостина Елена Петровна,
Харитоновна Анастасия Анатольевна

Поток: 1.3

Проверил:

Кулинич Ярослав Вадимович

Содержание

1. Введение	3
1.1 Назначение документа	3
1.2 Границы проекта (Scope)	3
2. Заинтересованные стороны и Акторы	3
2.1 Заинтересованные стороны	3
2.2 Акторы системы	4
1. Розничный Инвестор (User) – физическое лицо, использующее мобильное приложение, целью которого является приумножение капитала через покупку/продажу активов. Быстрое обновление цен, просмотр истории, оперативное удовлетворение заявок.	4
2. Агент нагрузочного тестирования – автоматизированный скрипт, имитирующий поведение пользователя, целью которого является создать нагрузку на систему, эквивалентную 10 000 пользователям.	4
3. Системный Администратор (Admin) – технический специалист, целью которого является мониторинг системы с помощью OpenTelemetry и логи ClickHouse.	4
3. Варианты использования (Use Cases)	4
3.1 Группа 1: Авторизация и Профиль	4
3.2 Группа 2: Рыночные данные (Real-time)	4
3.3 Группа UC-3: Торговые операции	4
4. Функциональные требования	5
4.1 Подсистема генерации данных (Low-Level)	5
4.2 Подсистема Бэкенда (Business Logic)	5
4.3 Подсистема Клиента (Android & Cross-platform)	6
5. Нефункциональные требования	6
5.1 Производительность и масштабируемость	6
5.2 Надежность и безопасность	6
5.3 Требования к коду и документированию	6
5.4 Отказоустойчивость и доступность	7
5.5 Удобство эксплуатации (Operability)	7
6. Требования к Среде и Инструментам	7

1. Введение

1.1 Назначение документа

Определение функциональных и нефункциональных требований к системе имитации биржевой торговли. Документ служит основой для разработки архитектуры, реализации кода и проведения приемочного тестирования.

1.2 Границы проекта (Scope)

Проект представляет собой высоконагруженный программный комплекс, эмулирующий деятельность брокерской компании.

Ключевые характеристики:

1. Поддержка высокой конкурентной нагрузки (до 10 000 одновременных сессий).
2. Гетерогенный технологический стек: Kotlin (Backend/Mobile), Go (Data Aggregation), C (Kernel Driver), React Native (Client UI).

В состав системы входят:

1. Серверную часть (бэкенд микросервисы).
2. Клиентские приложения (Mobile).
3. Генератор рыночных данных (драйвер уровня ядра).
4. Инфраструктуру мониторинга и аналитики.

2. Заинтересованные стороны и Акторы

2.1 Заинтересованные стороны

1. **Заказчик:** Контролирует соответствие реализации техническому заданию, устойчивость к нагрузкам, валидирует выбранный стек технологий и архитектурные решения.
2. **Команда разработки:** Реализует проект с микросервисной архитектурой и ИРС.
3. **Бизнес-владелец (Виртуальный):** Заинтересован в прибыли брокера и стабильности платформы, минимизации ухода клиентов к конкурентам.

2.2 Акторы системы

1. **Розничный Инвестор (User)** – физическое лицо, использующее мобильное приложение, целью которого является приумножение капитала через покупку/продажу активов. Быстрое обновление цен, просмотр истории, оперативное удовлетворение заявок.
2. **Агент нагрузочного тестирования** – автоматизированный скрипт, имитирующий поведение пользователя, целью которого является создать нагрузку на систему, эквивалентную 10 000 пользователям.
3. **Системный Администратор (Admin)** – технический специалист, целью которого является мониторинг системы с помощью OpenTelemetry и логи ClickHouse.

3. Варианты использования (Use Cases)

3.1 Группа 1: Авторизация и Профиль

- 1.1 Регистрация: Пользователь создает аккаунт, система создает запись в PostgreSQL, начисление стартового виртуального капитала.
- 1.2 Аутентификация: Вход по учетным данным, выдача JWT-токена для авторизации запросов.
- 1.3 Просмотр портфеля: Получение актуального состояния счета (свободные средства + рыночная стоимость активов).

3.2 Группа 2: Рыночные данные (Real-time)

- 2.1 Список инструментов: Отображение реестра доступных акций с текущей ценой и дельтой за сутки (в %).
- 2.2 Графическая аналитика: Визуализация истории изменения цены.
- 2.3 Получение уведомлений: Приложение обновляет цену на экране без перезагрузки страницы.

3.3 Группа UC-3: Торговые операции

- 3.1 Покупка акций: Резервирование средств, проверка лимитов, списание денег, зачисление актива.

3.2 Продажа акций: Проверка наличия актива в портфеле, списание актива, зачисление денежных средств.

4. Функциональные требования

4.1 Подсистема генерации данных (Low-Level)

1. Драйвер устройства (Linux Kernel Module / C):
 - Регистрация символьного устройство в пространстве /dev.
 - Генерация псевдослучайных последовательностей, имитирующих волатильность цены (price delta).
 - Предоставление интерфейса read() для чтения данных из User Space.
2. Агрегатор котировок (Go Service):
 - Опрос драйвера в бесконечном цикле с минимальными задержками.
 - Сериализация данных в формат JSON-объект тикера: {ticker, price, timestamp}.
 - Публикация событий в шину сообщений (Redis Pub/Sub).

4.2 Подсистема Бэкенда (Business Logic)

1. Message Broker (Redis/KeyDB):
 - Обеспечение доставки сообщений с минимальной задержкой между Go-агрегатором и Kotlin-сервисами.
 - Кэширование “последней известной цены” для ускорения REST-ответов.
2. API Gateway (Kotlin + Ktor):
 - Поддержка долгоживущих WebSocket соединений для трансляции котировок.
 - Предоставление REST endpoints для торговых операций.
 - Валидация запросов (проверка JWT).
3. Сервис Персистентности (Kotlin + PostgreSQL):
 - Гарантия ACID-транзакций при изменении балансов.
 - Запрет отрицательного баланса и продажи отсутствующих активов.
4. Сервис Аналитики (ClickHouse):
 - Персистентность каждого изменения цены для построения исторических графиков.
 - Агрегация пользовательских логов для аудита действий.

4.3 Подсистема Клиента

4.3.1 Стандартное приложение (React Native)

Назначение: Базовое приложение для розничных инвесторов.

Интерфейс (UI):

- Главный экран: Список инструментов с текущими ценами
- Детализация: График цены, кнопки "Купить"/"Продать"
- Портфель: Круговая диаграмма активов, текущий баланс

Сетевое взаимодействие:

- REST-клиент для торговых операций
- WebSocket-клиент для получения котировок в реальном времени

4.3.2 Терминал (Jetpack Compose)

Сетевое взаимодействие:

- REST-клиент (Ktor) для операций
- WebSocket-клиент с поддержкой сжатия данных
- Локальное кэширование данных

5. Нефункциональные требования

5.1 Производительность и масштабируемость

- Пропускная способность: Система должна выдерживать нагрузку 10 000 активных клиентов (10 тысяч открытых WebSocket соединений).
- End-to-End задержка доставки котировки (от Генератора до UI клиента) не должна превышать 1000 мс.

5.2 Надежность и безопасность

- Целостность данных: Все финансовые операции должны быть атомарными. Недопустимо частичное выполнение транзакции.
- Хранение секретов: Пароли пользователей должны храниться в виде хэшей.

5.3 Требования к коду и документированию

- Документирование: Весь программный код должен сопровождаться комментариями в формате KDoc (для Kotlin) или Javadoc.

- Архитектурный стиль: Backend-сервисы должны следовать принципам Clean Architecture с четким разделением ответственности на слои:
 - Domain (Бизнес-логика и сущности).
 - Data (Репозитории, работа с БД).
 - Presentation/UI (Контроллеры, DTO).

5.4 Отказоустойчивость и доступность

- Доступность (Availability): Допустимое время недоступности системы — не более 0.1% в месяц ($Uptime \geq 99.9\%$).
- Изоляция сбоев: Падение одного микросервиса не должно приводить к каскадному отказу всей системы (Circuit Breaker pattern).
- Backup Strategy: Автоматическое резервное копирование основной БД (PostgreSQL) должно выполняться ежедневно с хранением бэкапов не менее 7 дней.

5.5 Удобство эксплуатации (Operability)

- Метрики: Критические компоненты (Message Broker, DB, Gateway) должны отдавать метрики в формате Open Telemetry.
- Логирование:
 - Формат: Структурированные логи (JSON).
 - Хранение: Централизованный сбор и хранение в ClickHouse.

6. Требования к среде и инструментам

Языки: Kotlin (JVM & Android), Go, C, JS/TS (React Native).

СУБД: PostgreSQL (Основная БД), ClickHouse (Аналитика/Логи), Redis (Кэш/Брокер)

Мониторинг: OpenTelemetry, Grafana/Prometheus (для визуализации метрик).

CI/CD & VCS: Git (GitHub/GitLab).

Документация: Markdown в репозитории, Obsidian (база знаний).